Université Grenoble Alpes – INF452 Logique 2026

Rapport du Projet

RESOLUTION DU JEU NORINORI

LANGUE UTILISE : PYTHON

SOLVER-SAT : MINISAT

FORMAT FICHIER CNF : DIMACS

NEMA VICTOR KOLIE

ABDOURAHMANE DIALLO

MOUAFO DJUIDJE PAULINE

Table des matières

1. Présentation du jeu Norinori : 2

1.1 Règles du jeu : 2

1.2 Exemples de grilles : 3

1.3 Contraintes et remarques stratégiques : 4

2. Traduction du problème en logique propositionnelle 4

2.1 Variables booléennes : 4

2.2 Règle 2 et 3 : 6

3. Modélisation sous forme normale conjonctive 8

4. Implémentation 8

4.1. ***Architecture du programme*** 8

4.2. ***Format d'entrée*** 9

4.3. ***Génération du fichier DIMACS*** 9

4.4. ***Appel au SAT-solveur MiniSat*** 10

4.5. ***Affichage de la solution*** 10

4.6. Jeux de tests 11

5. Conclusion : 11

1. Présentation du jeu Norinori :

Le Norinori est un jeu de logique inventé par la société japonaise Nikoli, également connue pour avoir popularisé le Sudoku.

Le principe est le suivant :

On dispose d'une grille rectangulaire divisée en zones (régions). L'objectif est de colorier certaines cases en noir en respectant les règles suivantes :

1.1 Règles du jeu :

Règle 1 - Exactement 2 cases noires par zone :

Chaque zone doit contenir exactement deux cases noires.

Cela signifie :

- Ni plus de deux
- Ni moins de deux

Règle 2 – Les cases noires doivent être adjacentes deux à deux :

Chaque case noire doit être adjacente orthogonalement (haut, bas, gauche, droite) à exactement une autre case noire.

Donc :

- Les cases noires forment uniquement des paires
- Pas de case noire isolée

Règle 3 – Pas d'adjacence multiple :

Une case noire ne peut pas toucher :

- Deux cases noires différentes de façon orthogonales.
- Les diagonales ne comptent pas comme adjacence valide.

1.2 Exemples de grilles :

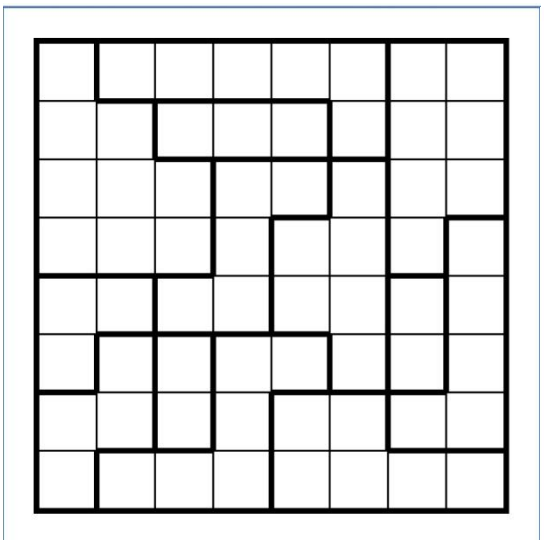
Une grille de Norinori est :

- Une grille rectangulaire (par exemple 6x6)
- Divisée en plusieurs zones
- Chaque zone ayant une taille variable

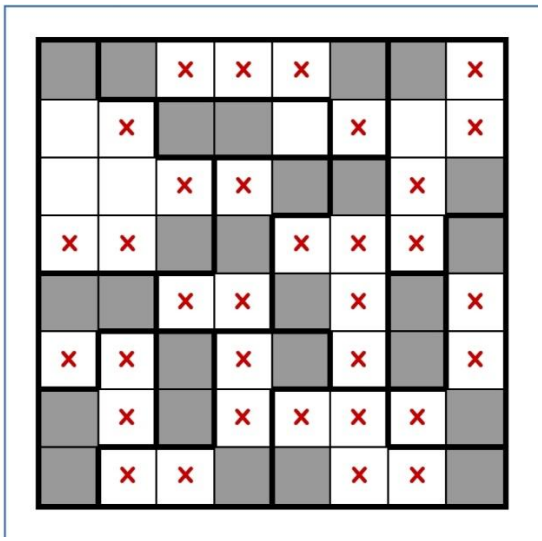
Exemple de grille avec solution :

Les cases avec une croix rouge sont des cases inutilisables. Il y a certaines cases inutilisables qu'on n'avait pas pu mettre des croix rouges car on avait déjà fini le jeu. Nous avons résolu ces grilles sur le https://fr.puzzle-norinori.com/. Après avoir résolu la grille, on a plus accès à la grille, c'est pourquoi on n'a pas pu mettre des croix rouges dans certaines cases inutilisables.

Grille 8x8

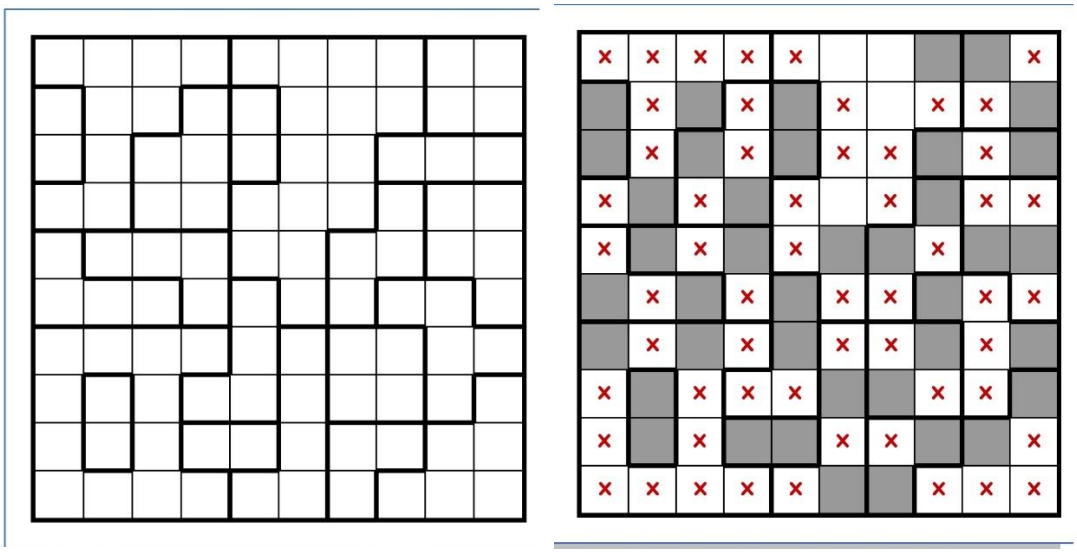


Solution



Grille 10x10

Solution



1.3 Contraintes et remarques stratégiques :

Certaines propriétés utiles à la résolution du problème :

- Une zone de taille 2 => les deux cases sont forcément noires.
- Une zone de taille 3 => une case sera forcément blanche.
- Si une case noire est placée, une de ses 4 voisines orthogonales doit être noire.
- Toutes les cases adjacentes aux cases coloriées deviennent donc inutilisables car ne respectent plus la règle N°3
- Toute case entièrement entourée de cases inutilisables devient elle-même inutilisable.
- Dans une zone où il ne reste que deux cases possibles, ces deux cases doivent être coloriées.
- Les cases en bordures sont plus contraintes car certaines de leurs voisines sont hors de la grille et donc ne comptent pas (ça se jouera sur les indices i.e. i – 1 et j –1 sont supérieurs ou égaux à 1 ; i +1 est inférieur ou égal à n et j +1 est inférieur ou égal à m avec n l'indice de ligne et m l'indice de colonne).

2. Traduction du problème en logique propositionnelle

L'objectif est de transformer le jeu Norinori en un problème SAT, c'est-à-dire un ensemble de formules en logique propositionnelle.

2.1 Variables booléennes :

Soit une grille de taille n x m.

Pour chaque case (i, j), on introduit une variable propositionnelle :

$x_{i,j}$: “La case (i, j) est coloriée (noire)“. Alors $\neg x_{i,j}$ signifie “la case (i, j) n’est pas coloriée“.

<u>Règle 1 – Exactement 2 cases noires par zone :</u>

On veut : exactement 2 cases coloriées dans la zone, c’est-à-dire :

$$Exactement2(Z) = AuMoins(Z) \wedge AuPlus(Z)$$

<u>Au Moins 2 :</u>

On souhaite garantir qu’il y ait au moins 2 cases coloriées dans chaque zone. Pour cela, on interdit qu’il y ait 0 ou 1 case coloriée.

Les clauses suivantes :

Pour chaque case C_i, on a :

$$\bigwedge_{i=1}^{n} (\neg C_i \bigvee_{j \neq i}^{n} C_j)$$

Interdisent qu’une case soit la seule coloriée.

$$C_1 \vee C_2 \vee \ldots \vee C_n$$

Cependant, elles n’empêchent pas que toutes les cases soient fausses.

Pour interdire cela, on ajoute donc la clause :

afin d’imposer qu’au moins une case soit coloriée.

L’ensemble garantit alors qu’il y a au moins deux cases coloriées.

C’est-à-dire :

$$(\bigwedge_{i=1}^{n} \neg C_i \bigvee_{j \neq i}^{n} C_j) \wedge (C_1 \vee C_2 \vee \ldots \vee C_n)$$

<u>Au plus 2 :</u>

On veut interdire que 3 cases ou plus soient vraies en même temps. La méthode utilisée est d’interdire la véracité à la fois de

chaque triplet (intersection des trois variables qui représentent trois cases).

Pour {A, B, C, D}, les triplets sont : {A, B, C}, {A, B, D}, {A, C, D}, {B, C, D}

$\neg(A \wedge B \wedge C) \equiv \neg A \vee \neg B \vee \neg C$

$\neg(A \wedge B \wedge D) \equiv \neg A \vee \neg B \vee \neg D$

$\neg(A \wedge C \wedge D) \equiv \neg A \vee \neg C \vee \neg D$

$\neg(B \wedge C \wedge D) \equiv \neg B \vee \neg C \vee \neg D$

Ainsi, nous obtenons finalement :

$$\bigwedge_{\substack{i,j,k=1 \\ i\neq j \\ i\neq k \\ j\neq k}}^{n} (\neg C_i \vee \neg C_j \vee \neg C_k)$$

Ainsi pour la règle 1 nous obtenons la formule suivante :

Règle 1 = ''Au moins 2'' ET ''Au plus 2''

Donc :

$$\left(\left(\bigwedge_{i=1}^{n} (\neg C_i \bigvee_{i\neq j}^{n} C_j) \wedge (C_1 \vee C_2 \vee \ldots \vee C_n) \right) \wedge \left(\bigwedge_{\substack{i,j,k=1 \\ i\neq j \\ i\neq k \\ j\neq k}}^{n} (\neg C_i \vee \neg C_j \vee \neg C_k) \right) \right)$$

2.2 Règle 2 et 3 :

- Règle 2 : chaque case coloriée fait partie d'un domino de deux cases adjacentes orthogonalement.
- Règle 3 : Deux dominos ne se touchent pas orthogonalement (diagonalement c'est possible).

Ensemble, ces deux règles reviennent à imposer :

« Chaque case coloriée a exactement un voisin orthogonal colorié. »

On modélise pour une case $x_{i,j}$

Ses voisins orthogonaux possibles sont $x_{i+1,j}$, $x_{i-1,j}$, $x_{i,j+1}$, $x_{i,j-1}$ – selon les bords.

La contrainte : si $x_{i,j}$ est coloriée, elle a exactement 1 voisin colorié.

On traduit « exactement 1 » = ''au moins 1'' ET ''au plus 1''.

<u>Au moins 1 voisin colorié (si coloriée) :</u>

$$x_{i,j} \Rightarrow x_{i+1,j} \vee x_{i-1,j} \vee x_{i,j+1} \vee x_{i,j-1}$$
$$\equiv \neg x_{i,j} \vee x_{i+1,j} \vee x_{i-1,j} \vee x_{i,j+1} \vee x_{i,j-1}$$

<u>Au plus 1 voisin colorié :</u>

On interdit que 2 voisins soient coloriés en même temps, pour chaque paire de voisins :

$x_{i,j} \Rightarrow \neg(x_{i+1,j} \wedge x_{i-1,j})$

$x_{i,j} \Rightarrow \neg(x_{i+1,j} \wedge x_{i,j+1})$

$\ldots etc. pour\ chaque\ paire$

$\equiv$

$\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i-1,j})$

$\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i,j+1})$

$\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i,j-1}$

$\neg x_{i,j} \vee \neg x_{i-1,j} \vee \neg x_{i,j+1}$

$\neg x_{i,j} \vee \neg x_{i-1,j} \vee \neg x_{i,j-1}$

$\neg x_{i,j} \vee \neg x_{i,j+1} \vee \neg x_{i,j-1}$

Etant traduit comme : « exactement 1 » = ''au moins 1'' ET ''au plus 1'', nous obtenons ainsi la formule suivante :

$$[\neg x_{i,j} \vee x_{i+1,j} \vee x_{i-1,j} \vee x_{i,j+1} \vee x_{i,j-1}] \wedge [(\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i-1,j}) \wedge (\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i,j+1}) \wedge (\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i,j-1}) \wedge (\neg x_{i,j} \vee \neg x_{i-1,j} \neg x_{i,j+1}) \wedge (\neg x_{i,j} \vee \neg x_{i-1,j} \vee \neg x_{i,j-1}) \wedge (\neg x_{i,j} \vee \neg x_{i,j+1} \vee \neg x_{i,j-1})]$$

3. Modélisation sous forme normale conjonctive

Ainsi, nous regroupons ces différentes clauses pour avoir la forme normale conjonctive.

Donc finalement on a :

$$\left(\left(\bigwedge_{i=1}^{n} (\neg C_i \bigvee_{i \neq j}^{n} C_j) \wedge (C_1 \vee C_2 \vee \ldots \vee C_n) \right) \wedge \left(\bigwedge_{\substack{i,j,k=1 \\ i \neq j \\ i \neq k \\ j \neq k}}^{n} (\neg C_i \vee \neg C_j \vee \neg C_k) \right) \right.$$

$$\wedge \left(\left[\neg x_{i,j} \vee x_{i+1,j} \vee x_{i-1,j} \vee x_{i,j+1} \vee x_{i,j-1} \right] \right.$$
$$\wedge \left[(\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i-1,j}) \wedge (\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i,j+1}) \right.$$
$$\wedge (\neg x_{i,j} \vee \neg x_{i+1,j} \vee \neg x_{i,j-1}) \wedge (\neg x_{i,j} \vee \neg x_{i-1,j} \neg x_{i,j+1})$$
$$\left. \left. \wedge (\neg x_{i,j} \vee \neg x_{i-1,j} \vee \neg x_{i,j-1}) \wedge (\neg x_{i,j} \vee \neg x_{i,j+1} \vee \neg x_{i,j-1}) \right] \right)$$

4. Implémentation des programmes

4.1. Architecture du programme

Le programme est composé de 4 fichiers Python :

lecteur.py : lit et analyse le fichier de grille

generateur_dimacs.py : traduit la grille en clause CNF

affichage_solution.py : lit la sortie MiniSat et affiche la solution

main.py : coordonne l'ensemble du processus du début à la fin.

La première étape est assurée par le fichier **lecteur.py.** Ce module lit le fichier texte décrivant la grille Norinori et en extrait les dimensions, la matrice des zones et le dictionnaire associant chaque zone à ses cases.

La deuxième étape est assurée par generateur_dimacs.py. À partir des données extraites par le lecteur, ce module traduit les trois règles du jeu en clauses logiques et produit un fichier au format DIMACS (.cnf) prêt à être soumis au SAT-solveur. La troisième étape est l'appel au SAT-solveur MiniSat, effectué

automatiquement par main.py via une commande système. MiniSat lit le fichier DIMACS et produit un fichier de sortie (.sat) contenant soit la mention UNSAT si aucune solution n'existe, soit une assignation de valeurs booléennes à chaque variable si une solution a été trouvée. La quatrième et dernière étape est assurée par affichage_solution.py. Ce module lit la sortie de MiniSat, identifie les variables vraies qui correspondent aux cases noires, reconstruit la grille et l'affiche de façon lisible à l'écran, accompagnée d'une vérification automatique du respect de la règle 1 pour chaque zone.

L'ensemble du pipeline est effectué par main.py, qui appelle chaque module dans l'ordre et gère les cas d'erreurs tels que l'absence de solution ou un fichier d'entrée introuvable.

4.2. ***Format d'entrée***

La grille est décrite dans un fichier texte (.txt) :

- Ligne 1 : '' n m '' (dimensions de la grille)
- Lignes suivantes : identifiants de zone séparés par des espaces

Exemple :

5 6

A A A B B B

A A C C B B

D D C C E E

D D F F E E

G G F F H H

4.3. ***Génération du fichier DIMACS***

Chaque case (i, j) de la grille est associée à une variable SAT par la formule :

var (i, j) = i * m + j + 1

Le +1 est obligatoire car les variables SAT commencent à 1, jamais à 0.

Exemple sur une grille 5×6 (m = 6) :

(0,0) →1 (0,1) →2 (0,2) →3 (0,3) →4 (0,4) →5 (0,5) →6

(1,0) →7 (1,1) →8 (1,2) →9 (1,3) →10 (1,4) →11 (1,5) →12

...

(4,0) →25 (4,1) →26 (4,2) →27 (4,3) →28 (4,4) →29 (4,5) →30

Les clauses générées correspondent aux 3 règles formalisées en section 2 et 3. Le fichier DIMACS a le format :

p cnf nb_variables nb_clauses

clause1_litteral1 clause1_litteral2 … 0

…

Exemple pour une grille de taille 5x6 :

p cnf 30 238

4.4. ***Appel au SAT-solveur MiniSat***

MiniSat est appelé automatiquement via la commande :

minisat instance.cnf instance.sat

Code de retour :

10 = SAT (solution trouvé)

20 = UNSAT (pas de solution possible)

4.5. ***Affichage de la solution***

La sortie MiniSat est un fichier contenant :

SAT

-1 -2 3 4 -5 … 0

Les entiers positifs correspondent aux cases noires. Le programme reconstruit et affiche la grille :

■ = case noire □ = case blanche

4.6. Jeux de tests

Les jeux de tests se trouvent dans le dossier instance du projet. Nous y avons :

- Grille 1x1 (une zone et une case) Pas de solution
- Grille 2x2 (1 zone 4 cases)
- Grille 2x2 (2 zones et une zone avec une seule case) Pas solution
- Grille 5x6 (8 zones)
- Grille 4x4 (4 zones)
- Grille 20x20 (très large)

5. Conclusion :

Ce projet nous a permis de mettre en pratique la notion de réduction d'un problème combinatoire en un problème de satisfaisabilité (SAT).

Le jeu Norinori, bien que simple à comprendre, a nécessité une modélisation rigoureuse. La principale difficulté a été la traduction des règles 2 et 3, qui imposent qu'une case noire ait exactement un voisin orthogonal noir. Cette contrainte "exactement 1" a dû être décomposée en deux parties distinctes : "au moins 1 voisin" et "au plus 1 voisin", chacune donnant lieu à des clauses CNF différentes.

Le SAT-solveur MiniSat s'est montré remarquablement efficace. Même sur une grille 20×20 générant 400 variables et 3906 clauses, la résolution s'effectue en moins d'une seconde. Cela confirme que l'approche SAT est bien adaptée à ce type de problème combinatoire.